Module 5 — Software Assurance Report

Grad Cafe Analytics: Security Hardening & Reproducible Environment

---

1. Installation & Reproducible Environment

The project is fully installable as a Python package via **pip + venv** or the newer **uv** tool. Both methods produce an identical, reproducible environment from *requirements.txt* and *setup.py*.

Using pip + venv:

```
python -m venv .venv
.venv\Scripts\activate        # Windows
source .venv/bin/activate    # Linux/macOS
pip install -r requirements.txt
pip install -e .
```

Using uv (alternate install path):

```
uv venv
uv pip sync requirements.txt
uv pip install -e .
```

Why setup.py? Adding *setup.py* makes the project installable as a package, ensuring consistent import paths across local runs, tests, and CI. It enables editable installs (*pip install -e .*) which eliminate 'it works on my machine' path issues. Tools like uv can also extract dependencies from setup.py when syncing environments, making the project portable and reproducible across any clean Python environment.

Running the Flask App:

```
cd src
python app.py
# Visit http://localhost:8080
```

2. Dependency Graph Analysis

Generated with:

```
pydeps src/app.py --noshow -T svg -o dependency.svg
```

The dependency graph reveals the full import structure of the Grad Cafe Analytics project. **app.py** serves as the central hub, importing from **query_data**, **clean**, and **load_data** to orchestrate the full

data pipeline through the Flask web interface. **db_config** acts as a shared foundation, imported by both **load_data** and **query_data** to provide a single, consistent database connection resolved entirely from environment variables. **clean.py** depends on standard library modules (re, json, pathlib) and performs all ETL transformations without any database coupling, keeping it independently testable. **scrape.py** is notably isolated from the rest of the application — it only imports from standard library and third-party packages (selenium, beautifulsoup4, requests), with no internal dependencies, meaning it can be replaced or mocked without affecting any other module. **load_data** connects the scraping and cleaning pipeline to the database, importing from both **db_config** and consuming cleaned records. The graph confirms a clean layered architecture: scrape → clean → load → query → app, with no circular dependencies.

3. SQL Injection Defenses

All SQL queries in *query_data.py* were refactored to use **psycopg2 SQL composition** rather than string concatenation, f-strings, or .format() calls. This ensures user-supplied values are never interpolated directly into SQL text.

What Changed:

Technique	Before	After
Dynamic fields	f-string or + concat	sql.Identifier(column_name)
User values	Embedded in SQL text	%s with params tuple
Query structure	Plain strings	sql.SQL().format()
Result limits	No LIMIT clause	LIMIT 1 or LIMIT 100 on every query

Example — get_filtered_results() using safe composition:

```
stmt = sql.SQL(
    "SELECT {fields} FROM {table} WHERE term ILIKE %s LIMIT %s"
).format(
    fields=sql.SQL(", ").join([sql.Identifier("program"), ...]),
    table=sql.Identifier("applicants"),
)
cur.execute(stmt, (f"%{term}%", limit))
```

Why this is safe: sql.Identifier() properly quotes table and column names, preventing SQL injection through dynamic identifiers. User-supplied values (term, limit) are passed as a separate params tuple and never concatenated into the SQL string — psycopg2 handles escaping at the driver level. Every query includes a LIMIT clause clamped between 1 and 100 via *max(1, min(int(limit), MAX_LIMIT))*, preventing unbounded result abuse even if a malicious limit value is supplied.

4. Least-Privilege Database Configuration

The application connects to PostgreSQL using a dedicated least-privilege account **gradcafe_user** rather than the default superuser. This limits the blast radius of any SQL injection or credential compromise.

SQL used to create and configure the account:

```
CREATE USER gradcafe_user WITH PASSWORD 'yourpassword';
GRANT CONNECT ON DATABASE gradcafe TO gradcafe_user;
GRANT USAGE ON SCHEMA public TO gradcafe_user;
GRANT SELECT ON TABLE applicants TO gradcafe_user;
```

Permissions Granted and Rationale:

Permission	Granted?	Reason
CONNECT on database	Yes	Required to open any connection
USAGE on schema public	Yes	Required to access tables in the schema
SELECT on applicants	Yes	App is read-only — only reads query results
INSERT / UPDATE / DELETE	No	App never writes — not needed
DROP / ALTER / TRUNCATE	No	Would allow destructive schema changes
SUPERUSER	No	Would bypass all access controls
CREATE TABLE / INDEX	No	Schema is managed separately by load_data.py

Credentials management: No credentials are hard-coded anywhere in the source code. The application reads *DB_HOST*, *DB_PORT*, *DB_NAME*, *DB_USER*, and *DB_PASSWORD* (or *DATABASE_URL*) from environment variables at runtime. A *.env.example* file documents the required variables with placeholder values. The real *.env* file is excluded from version control via *.gitignore*, ensuring secrets are never committed to the repository.

5. Static Analysis — Pylint 10/10

Command used:

```
pylint src/app.py src/clean.py src/db_config.py src/load_data.py src/query_data.py
src/scrape.py
```

All six source files achieve a final Pylint score of **10.00/10** with zero warnings or errors. Key improvements made: extracted helper functions to reduce local variable counts, added docstrings to all public functions, fixed f-strings without interpolation, resolved redefined-outer-name issues, split long lines, and added OS-aware signal handling for Chrome process termination (SIGKILL on Linux, SIGILL on Windows).